Autonomous Preservation Robots in an Interplanetary Museum Vault

Seyed Morteza Mojtabavi, seyed.mojtabavi@unitn.it, 256328

Date: March 21, 2026

University of Trento
Department of Information Engineering and Computer Science
Via Sommarive 9, 80123
Povo (TN), Italy

1 Scenario Understanding and Justification

This project addresses the Interplanetary Museum Vault (IMV) space scenario. The IMV is a subterranean Martian structure storing fragile artifacts from various space missions. Due to seismic activity and pressure fluctuations, an autonomous robotic curator must relocate high-value artifacts to a safe Stasis-Lab while managing environmental hazards and limited resources.

Scenario Choice Justification: The space scenario was selected because it naturally introduces complex, multi-layered constraints. These include topological hazards (low-pressure zones), strict sequential processing requirements (cooling temperature-sensitive items), and finite physical resources (Anti-Vibration Pods). These constraints provide a robust testbed for comparing how different planning paradigms (Classical, Hierarchical (HTN), and Temporal) handle resource bottlenecks and negative preconditions.

2 Design Choices and Technical Approach

The domain was iteratively expanded from PDDL 2.1 Level 1 (STRIPS/Typing) to HDDL (HTN) and PDDL 2.1 Level 3 (Durative Actions).

2.1 Modeling Strategies

- **Type Hierarchy:** I utilized strict typing (location, agent, spot, artifact, pod) with robot and drone sub-typed under agent. This reduces the number of ground actions and prevents parameter errors during grounding.
- **The Slot System:** Agent capacity is modeled using discrete spot objects linked to agents via the belongs-to predicate. This allows for dynamic capacity (for example, the curator has 2 spots, the drone has 1) without hardcoding separate actions for different loads.
- **Positive Antonyms:** In the temporal domain (Problem 4), I replaced negative preconditions with positive antonyms like normal-pressure and sealing-off. This ensures compatibility with temporal planners like Optic and TFD, which can struggle with intervals containing negative conditions.
- **Concurrency Mutex:** I introduced an available predicate for durative actions. This acts as a mutex lock, requiring an agent to be available at start and restoring it at end, preventing a single agent from performing overlapping tasks while allowing multi-agent parallelization.

2.2 Action and Method Design

- **Universal Quantification (forall):** The unload-pod action uses a forall check to ensure every spot belonging to a robot is an empty-spot before dropping the pod. This prevents the physical paradox of a robot dropping protective gear while still carrying a delicate artifact.
- **HTN Task Decomposition:** In HDDL (Problem 3), methods were structured to enforce specific workflows. For example, the evacuate-hot task forces the agent to visit the cryo-chamber before the stasis-lab. This embeds specific domain knowledge directly into the planner, bypassing the need for it to guess the correct topological order.

3 Assumptions and Deviations

- **Artifact Finality:** It is assumed that once an artifact is deposited in the Stasis-Lab, it is permanently secured. The unload-artifact action removes the "carried" status but does not restore the free-artifact status, preventing redundant pick-up loops.
- **Separation of Loading Logic:** Loading actions were bifurcated into load-normal-artifact and load-delicate-artifact. This deviation cleanly enforces the rule that delicate items must have an anti-vibration pod without complicating the preconditions for standard items.

4 Comprehensive Experimental Results

The following data reflects the results obtained when solving the formulated problems across various planners and configurations.

4.1 Classical Planning (Problems 1 and 2)

Testing was performed using Fast Downward via Planutils.

Problem	Configuration	Time (s)	Status	States Exp.	Observation
P1	LAMA-First	0.26s	Satisficing	53	Valid plan found quickly.
P1	A* (Blind)	0.29s	Optimal	111,651	Guaranteed shortest path.
P1	seq-opt-lmcut	N/A	Failed	N/A	Exit code 34; neg. preconditions.
P2	LAMA-First	0.32s	Satisficing	53	Produced suboptimal sequence.
P2	A* (Blind)	0.39s	Optimal	111,651	Optimal path found.

4.2 Hierarchical Task Networks (Problem 3)

Testing was performed using the PANDA framework.

Algorithm	Heuristic	Length	Nodes	Total Time	Status
PANDA (A*)	hhRC(hFF)	40 steps*	394	1.018s	Solved.

**Note: Plan length includes SHOP method precondition checks.*

4.3 Temporal Planning (Problem 4)

Testing was performed using Optic and Temporal Fast Downward (TFD).

Configuration	Total Time	Makespan	Evaluated	Observation
Optic (Greedy)	0.09s	85.008	66	Highly concurrent plan.
Optic (Weighted A*)	0.10s	85.008	66	Identical to Greedy search.
Optic (BFS)	12.47s	189.019	38,185	Suboptimal concurrency.
TFD (Anytime)	$> 300s$	N/A	$> 207,000$	State-space explosion.

5 Critical Discussion and Scalability Analysis

- **Search Space Complexity:** The results highlight a severe scalability bottleneck in purely declarative search. Merely adding one drone and two carrying spots in Problem 2 forced the A* planner to expand over 111,000 states to prove optimality.
- **HTN vs. Classical Efficiency:** Problem 3 demonstrated the power of hierarchical decomposition. By encoding domain knowledge into methods, search nodes dropped from over 111,000 to just 394, proving that HTNs scale linearly where classical search scales exponentially.
- **Temporal Heuristics:** The Optic Greedy search identified parallel opportunities (for example, the curator enabling sealing while the drone moves) instantly, whereas the TFD failure underscores that concurrent state spaces are too vast without specialized temporal heuristics.

6 PlanSys2 Implementation (Problem 5)

The temporal model was deployed in PlanSys2 using the plansys2-assignment ROS 2 package. Actions were implemented as C++ nodes (such as cooling-node.cpp) simulating durative execution via progress variables and feedback triggers. Final execution logs in the terminal confirmed that the PlanSys2 executor synchronized these nodes according to the generated plan while respecting agent availability mutexes.

7 Archive Organization

The project archive is structured as follows:

- **/project files/Problem1**: Baseline PDDL domain and problem.
  - /astar-blind: Optimal plan solution and execution screenshots.
  - /lama-first: Satisficing plan and performance metrics.
- **/project files/Problem2**: Extended PDDL for drones and capacity system.
  - /astar-blind and /lama-first: Multi-agent search logs and plans.
- **/project files/Problem3**: HDDL files and HTN solver outputs.
  - /Panda: Full execution trace and sequence from the PANDA planner.
- **/project files/Problem4**: Temporal PDDL and comprehensive solver logs:
  - /Optic BFS, /Optic Greedy, /Optic Weighted A-Star.
  - /TFD Anytime and /TFD Basic.
- **/project files/Problem5**: The complete ROS 2 infrastructure for deployment.
  - /plansys2-assignment: Full package with CMakeLists.txt, launch scripts, and C++ source code.